

Schema Alignment solution using Large Language Models

Project for Big Data Integration course

Alessandro Di Girolamo

Project repository: <https://github.com/aledigirm3/advanced-topics-computer-science/tree/main/BigData-Integration>

Keywords: LLM, Big Data Integration, Computer Science, Schema Alignment, Prompt Engineering

1. Introduction

This report aims to present the analysis conducted and the approach taken to address a Big Data Integration problem. Specifically, it focuses on schema alignment through the use of Large Language Models and prompt engineering techniques. Additionally, potential alternative solutions and challenges that could enhance the final evaluation results are briefly discussed as future considerations.

2. Problem Analysis

The task at hand involves the development of a system that, given a natural language query, can determine which tables are necessary to fulfill the query's requirements, using the BIRD benchmark.

The primary challenge with this dataset is that the queries are based on databases from a wide range of domains, requiring a deep understanding of the semantics of the queries to effectively narrow the scope of interest and perform increasingly fine-grained analyses. The use of Large Language Models is certainly a valid approach, but several factors must be considered, including token costs, model selection, prompt optimization, and potential variability in the model's responses.

The following sections outline the various approaches employed for this task.

3. Methodology

In this section, the various approaches employed will be presented, aiming to break the problem down into two subproblems to facilitate a more effective solution.

3.1 Databases extractions

The first subproblem involves processing the natural language query and extracting the three databases most similar to it, thereby mitigating the challenge posed by the large number of databases spanning vastly different domains. The underlying assumption is that databases with names highly unrelated to the query’s context will exhibit very low similarity to the query itself.

This approach has been implemented using two different techniques: the first leverages a Large Language Model (LLM), while the second utilizes a locally hosted sentence embedder.

3.1.1 LLM solution

For the solution utilizing Large Language Model, *llama-3.3-70b-versatile* was employed through the APIs provided by the *Groq* platform.

In this phase, prompt engineering played a crucial role. A system prompt was designed to include context specifications (effectively ‘specializing’ the model) along with a list of all databases and example queries/responses to serve as references.

Once the system prompt was prepared, the model processed the queries individually, and the responses were stored. The primary limitation of this approach was the token limit, which significantly slowed down the overall process.

3.1.2 Sentence Embedder solution

This approach was designed to mitigate token costs, which are limited on Groq, by leveraging a sentence embedder, specifically the *all-MiniLM-L6-v2* model.

Embeddings were generated for all database names and queries. The three most similar databases for each query were then identified using cosine similarity.

3.1.3 Considerations

Before proceeding to the second part of the solution, is important to consider the chosen approach for the final evaluation, considering the performance of the two methods described below

	llama@3	all-MiniLM@3	all-MiniLM@4
Accuracy	0.97	0.74	0.81

Table 1: Databases Extractors accuracy

It can be observed that the accuracy of LLM is significantly higher than sentence embedder. In most cases, the LLM successfully includes the relevant database among the three identified.

Nevertheless, the results obtained from the sentence embedder remain valid. Their performance could be significantly improved by exploring alternative approaches, such as incorporating table names and attributes into the database embeddings.

3.2 Tables extractions

The second subproblem involves reprocessing the query, but this time considering only the databases identified in the first step. Specifically, llama-3.3-70b-versatile was again employed and, similar to the database extraction phase, prompt engineering was applied. The system prompt was designed by providing the relevant context, defining the task, and incorporating a series of examples to further specialize the model. These examples consisted of a natural language query along with the three potential databases and their associated tables. The model’s responses were expected to include a single database and only the tables strictly necessary to fulfill the query.

Additionally, a variant of this approach was introduced, where table attributes were also provided along with the tables. This allowed for an analysis of the model’s performance in selecting necessary attributes and enabled a comparison of the two approaches based solely on table selection.

4. Results

The overall system was evaluated on a substantial portion of the dev dataset from BIRD benchmark, selecting a significant number of examples from each database, for a total of 1072 instances.

For the evaluation, regular expressions were used to extract the names of the tables required for the natural language queries from the SQL queries provided by BIRD.

The results of the experiment are presented below.

	Precision	Recall	F1-score
Table	0.71	0.79	0.75

Table 2: Performance metrics of solution

For the variant that includes table attributes, the performance results for both table selection and attribute selection are as follows.

The ground truth is based on 57 balanced examples across all databases.

	Precision	Recall	F1-score
Table	0.83	0.93	0.88
Attributes	0.65	0.83	0.73

Table 3: Performance metrics of solution with attributes

It can be observed that the results have improved in this case, demonstrating how the inclusion of attributes brings the model’s responses much closer to natural language.

The results obtained from LLM were, in very rare cases, different from what was specified in the prompt. These instances were identified and discarded through a quick search in the result files, which required only a few minutes of manual review. Given the size of the dataset, this additional verification step was considered an acceptable cost.

5. Conclusion

In conclusion, the results obtained are acceptable and demonstrate the power of LLMs in data integration tasks like this one.

However, it is important to consider that, for practical and time-related reasons, only one model was tested. Although this model is strong and reliable, experimenting with more advanced models could even achieve better results, though at a higher computational cost. Another possible improvement could be the further refinement of the prompt to observe how results change, which would certainly increase costs during various attempts, but could lead to significant improvements.

A future experiment will involve extending the ground truth to assess whether the performance degrades as the dataset size increases or if it remains stable.